Innovative Evaluation Practices for BCSE308P - Computer Networks Lab

Fall Semester 2026-27

CONTENDER-ESTIMATION ADAPTIVE BACKOFF (CEAB): A COMPARATIVE STUDY OF WAITING STRATEGIES FOR IEEE 802.11 CSMA/CA

23BCE0298	**SSUNVYA JAIN**
24BDS0233	**RISHIKA SINGAL**
24BDS0256	**SAHIL BARNWAL**

B.Tech.

in

Computer Science and Engineering

(Specialization if any)

School of Computer Science and Engineering

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

ABSTRACT

IEEE 802.11 stations share the wireless medium using CSMA/CA, and the binary exponential backoff (BEB) of the Distributed Coordination Function decides how long each station waits before transmitting. BEB doubles the contention window after a collision but returns it to the minimum after every success, so in a crowded cell every new frame starts in a window that is far too small. This project studies nine waiting strategies — BEB, constant, linear, Fibonacci, quadratic, MILD, EIED, Idle Sense and a new scheme, Contender-Estimation Adaptive Backoff (CEAB) — in a purpose-built, event-driven 802.11g DCF simulator that was validated against Bianchi's analytical model (throughput error below 1.7 %). CEAB lets every station overhear the transmitter address of successfully decoded frames and compute the effective number of contenders as the inverse Simpson index of the last 200 senders. An idle-gap load factor corrects the estimate under unsaturated traffic, and the window is set to the analytically optimal value for that number of stations instead of being reset to CWmin. Over 2170 simulation runs (10 seeds per configuration), CEAB raised saturation throughput by 15.8 % with 20 stations and 33.5 % with 50 stations, cut the 99th-percentile access delay by 79 %, lowered the collision probability from 0.59 to 0.20, raised short-term Jain fairness from 0.40 to 0.91, and stayed within 2.5 % of the theoretical throughput limit from 5 to 100 stations. Its estimator tracks sudden joins and departures within 150 ms. The evaluation also shows where the scheme gains little: with RTS/CTS, at high frame-error rates and at medium unsaturated load, and when it shares the channel with legacy BEB stations, which keep a larger share of the airtime.

Keywords: CSMA/CA, IEEE 802.11, DCF, backoff algorithms, contention window, contender estimation, MAC-layer simulation, fairness.

CERTIFICATE

This is to certify that the project report titled “Contender-Estimation Adaptive Backoff (CEAB): A Comparative Study of Waiting Strategies for IEEE 802.11 CSMA/CA” is a bonafide record of the work done by Ssunvya jain (23BCE0298), Rishika Singal (24BDS0233), SAHIL BARNWAL (24BDS0256) in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

Name and Signature of the Students

Ssunvya jain (23BCE0298) _____

Rishika Singal (24BDS0233) _____

SAHIL BARNWAL (24BDS0256) _____

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**ii**
1.	**INTRODUCTION**	**1**
	1.1 Background and motivation	1
	1.2 Problem statement	1
	1.3 Objectives	2
	1.4 Scope and limitations	2
	1.5 Mapping to course modules and the lab experiment	2
2.	**LITERATURE SURVEY**	**4**
	2.1 Overview of the existing protocol or scheme	4
	2.2 Review of related work	4
	2.3 Research gap and justification of novelty	5
3.	**PROPOSED SYSTEM**	**6**
	3.1 System architecture and topology	6
	3.2 Baseline mechanism description	7
	3.3 Proposed mechanism: algorithm, pseudocode, flowchart	8
	3.4 Novelty statement	10
	3.5 Assumptions and design decisions	10
4.	**IMPLEMENTATION**	**12**
	4.1 Tools, platform, versions and hardware	12
	4.2 Simulation setup	12
	4.3 Code structure and explanation of key modules	12
	4.4 Challenges faced and how they were solved	13
5.	**EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION**	**15**
	5.1 Performance metrics and why they were chosen	15
	5.2 Scenarios and parameter values	15
	5.3 Results: baseline vs proposed	15
	5.4 Analysis and discussion	22
	5.5 Critical evaluation	25
6.	**CONCLUSION AND FUTURE WORK**	**27**
	6.1 Summary of work and key findings	27
	6.2 Limitations	27
	6.3 Future enhancements	27
7.	**REFERENCES**	**29**
	Appendix A: Important source code listings	31
	Appendix B: Screenshots, event traces and raw result tables	35

1. INTRODUCTION

1.1 Background and motivation

Wi-Fi is the access network most people use every day, and in an IEEE 802.11 basic service set all stations share one radio channel. Because a radio cannot listen while it transmits, 802.11 cannot detect collisions the way Ethernet does; it avoids them instead. The mandatory Distributed Coordination Function (DCF) implements carrier-sense multiple access with collision avoidance (CSMA/CA): a station that wants to send waits until the medium has been idle for a DIFS, then counts down a random number of idle slots drawn from its contention window W, and transmits when the counter reaches zero. The size of W is the single most important knob of the MAC layer. If W is too small, several stations pick the same slot and their frames collide; if W is too large, the channel sits idle while everybody waits.

The standard chooses W with binary exponential backoff (BEB), an idea inherited from Ethernet [10]: W starts at CWmin+1 = 16 slots, doubles after every failed attempt up to CWmax+1 = 1024, and is reset to 16 after every success. Classic analysis [2] shows that this rule works well for a handful of stations but loses a growing share of the capacity as the number of contenders rises. Dense deployments — lecture halls, hostels, offices and IoT networks with dozens of active stations per access point — are exactly the case BEB handles worst. Many alternative ways of waiting have been proposed (linear, Fibonacci, polynomial, multiplicative-increase/linear-decrease, exponential-increase/exponential-decrease and feedback-controlled schemes), but they are usually evaluated in isolation, with different tools and assumptions. This project compares them side by side and proposes a new scheme.

1.2 Problem statement

Under BEB, the contention window reflects only the recent history of a single frame. Every successful transmission throws that history away, so a station in a crowded cell starts each new frame with W = 16, collides with high probability, doubles, collides again and only then reaches a window that suits the actual number of contenders. The results are (i) a collision probability of about 0.59 and a throughput loss of 25 % with 50 stations, (ii) severe short-term unfairness, because the station that just succeeded has the smallest window and tends to win again (the capture effect), and (iii) very long tail delays for the stations that lost several rounds. The problem addressed in this project is: **how can a station choose a contention**

window that matches the real level of contention, using only information it can observe on the channel, without needing all other stations to run the same algorithm?

1.3 Objectives

1. Build and validate a slot-accurate, event-driven simulator of the IEEE 802.11g DCF with pluggable backoff strategies, saturated and Poisson traffic, channel errors, RTS/CTS and dynamic station membership.
2. Implement and compare nine waiting strategies: BEB (baseline), constant window, linear, Fibonacci, quadratic, MILD, EIED, Idle Sense, and the proposed CEAB.
3. Design CEAB, a backoff scheme that estimates the number of active contenders from overheard frame addresses and sets the window to the analytically optimal value for that number.
4. Evaluate all schemes with repeated independent runs (mean ± standard deviation, Welch t-tests) under low, medium and high load, and in sensitivity, churn, coexistence and ablation scenarios.
5. Analyse critically when and why the proposed scheme helps, and identify the conditions under which it fails or gives no benefit.

1.4 Scope and limitations

The study covers the MAC sublayer of a single 802.11g (ERP-OFDM, 54 Mb/s) basic service set in which every station hears every other station (one collision domain). Uplink traffic from 2 to 100 stations is modelled either as saturated (always backlogged) or as Poisson arrivals. Both basic access and the RTS/CTS handshake are supported, as are random frame errors. Out of scope are hidden terminals, capture at the receiver, multi-rate stations, 802.11e EDCA access categories, frame aggregation and transport-layer effects. The work is a simulation study; it was not deployed on real Wi-Fi hardware, because changing the backoff rule requires firmware access that commodity adapters do not offer.

1.5 Mapping to course modules and the lab experiment

Table 1.1: Mapping of the project to the course syllabus and lab work.

Course topic (BCSE308P / BCSE308L)	Where it appears in this project
Data link layer: MAC sublayer and multiple access	Core of the project: CSMA/CA contention, slotted backoff, collisions, retransmission and retry limits (Ch. 3)
Random access protocols: ALOHA, CSMA, CSMA/CD, CSMA/CA	BEB inherited from Ethernet CSMA/CD; constant-window scheme behaves like p-persistent CSMA (Ch. 2, 3)
Wireless LANs: IEEE 802.11 architecture, DCF, RTS/CTS, IFS	802.11g timing (slot, SIFS, DIFS), ACK timeout, RTS/CTS mode (Ch. 4.2, 5)

Course topic (BCSE308P / BCSE308L)	Where it appears in this project
Network performance: throughput, delay, jitter, fairness	Metrics in Ch. 5.1; Bianchi throughput model; Jain fairness index
Lab: simulation of MAC protocols and performance analysis	Event-driven simulator, repeated runs, graphs and statistical comparison of the baseline and the proposed scheme
Lab: programming network protocols (Python)	Modular Python package, CLI/GUI demo, unit tests (Ch. 4.3, Appendix A)

2. LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

In the 802.11 DCF [1] a station with a frame first senses the medium. After an idle DIFS (28 µs in 802.11g) it enters backoff: it draws an integer b uniformly from [0, W−1] and decrements it once per idle slot of 9 µs, freezing the counter whenever the medium becomes busy. When b reaches zero it transmits. The receiver answers a correct frame with an ACK after a SIFS; a missing ACK is treated as a collision. On a collision BEB sets W ← min(2W, CWmax+1) and retries, up to a retry limit of 7; on success or after a drop it sets W ← CWmin+1. With RTS/CTS the data frame is preceded by a short RTS/CTS exchange, so a collision wastes only the RTS duration. Bianchi [2] modelled this process as a Markov chain and showed that each station attempts with an approximately constant probability τ per slot. Throughput depends on τ and on the number of stations n, and is maximised when $\tau \approx 1/(n\cdot\sqrt{(Tc/2\sigma)})$, where Tc is the collision duration and σ the slot time. Since n is not known to the stations, BEB has no way to reach this optimum.

2.2 Review of related work

Table 2.1: Literature survey.

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
1	Kleinrock & Tobagi, 1975 [11]	CSMA (non-/p-persistent) analysis	Throughput–delay; analytical	Foundation of carrier sensing; shows the effect of the attempt probability	Fixed persistence; no adaptive backoff
2	Metcalfe & Boggs, 1976 [10]	Binary exponential backoff in Ethernet	Utilisation; prototype measurement	Simple, distributed, stable for small n	Designed for collision detection; resets after success
3	Bharghavan et al., 1994 [4]	MACAW: MILD backoff (×1.5 / −1) and window copying	Throughput, fairness; simulation	Reduces oscillation and unfairness of BEB	Linear decrease is very slow; window can lock at CWmax
4	Håstad et al., 1996 [15]	Polynomial backoff	Stability, delay; theoretical analysis	Polynomial backoff is stable where BEB may not be	Large delays; asymptotic results only
5	Bianchi, 2000 [2]	Markov model of 802.11 DCF	Saturation throughput; analysis + simulation	Accurate closed form; defines the optimal τ	Analysis only; does not change the protocol
6	Calì et al., 2000 [3]	Dynamic tuning: estimate n, use p-persistent optimal τ	Throughput; analysis + simulation	Gets close to the theoretical limit	Model-based estimator; all stations must run the scheme
7	Bianchi & Tinnirello, 2003 [7]	Kalman-filter estimate of n from collision probability	Estimation accuracy; simulation	Accurate, fast tracking of n	Assumes all stations run standard BEB; not used to set W
8	Song et al., 2003 [5]	EIED (×2 on failure, ÷2 on success)	Throughput, delay; analysis + simulation	Remembers contention between frames	Fixed factors; unfair under high load
9	Ni et al., 2003 [12]	Slow contention-window decrease	Throughput, delay; analysis + ns-2	Simple change; better throughput for large n	No estimate of n; tuning factor fixed
10	Heusse et al., 2005 [6]	Idle Sense: drive mean idle slots to an optimal target (AIMD)	Throughput, fairness; simulation + testbed	Near-optimal throughput, good fairness	Needs every station to run it; starves next to legacy stations

S.No	Authors, year	Technique	Metrics and tool	Merits	Gaps
11	Manaseer et al., 2006 [14]	Fibonacci backoff for MANETs	Throughput, delay; simulation	Slower growth than BEB, fewer idle slots	Still resets after success; degrades for large n

2.3 Research gap and justification of novelty

Two families of solutions exist. **History-based rules** (BEB, linear, Fibonacci, polynomial, MILD, EIED, slow decrease) change W by a fixed rule after each outcome. They are simple, but they never learn how many stations are actually competing, so their window is right only by accident. **Estimation-based schemes** (Calì et al., Bianchi & Tinnirello, Idle Sense) do aim at the optimum, but each depends on a model of how the *other* stations behave: Calì et al. invert a p-persistent model, the Kalman estimator assumes that everybody runs standard BEB, and Idle Sense works only when every station runs Idle Sense. Our experiments confirm two practical consequences. First, inverting an analytical model is fragile in practice, because real backoff counters are correlated and the measured idle time deviates from the i.i.d. model (Section 4.4). Second, a station running Idle Sense next to legacy BEB stations obtained only 0.89 Mb/s against 27.0 Mb/s for the same number of legacy stations (Section 5.4).

The gap is therefore an estimator of the number of contenders that is **model-free, uses only passively observable information, and does not care which backoff rule the other stations run**, combined with a window rule that remains correct when traffic is not saturated. CEAB fills this gap by counting the transmitter addresses of decoded frames — information already present in every 802.11 MAC header — and summarising them with the inverse Simpson index, a diversity measure from ecology [16], [17] that, as far as we know, has not been used for 802.11 backoff tuning. A second gap is methodological: no single study compares the common waiting strategies under identical conditions with repeated runs and statistical tests; this project provides such a comparison for nine strategies.

3. PROPOSED SYSTEM

3.1 System architecture and topology

Figure 3.1 shows the simulated topology: one access point and N stations within range of each other, all sending uplink frames. Figure 3.2 shows the software architecture. Each station has a traffic source, a FIFO queue, the DCF state machine (backoff counter, retry counter) and a pluggable backoff strategy object that owns its contention window. A shared channel engine decides, slot by slot, whether a transmission succeeds, collides or is corrupted, charges the corresponding airtime, and tells every station the length of the idle gap and the transmitter address (TA) of each decoded frame — exactly what a real 802.11 radio can observe.

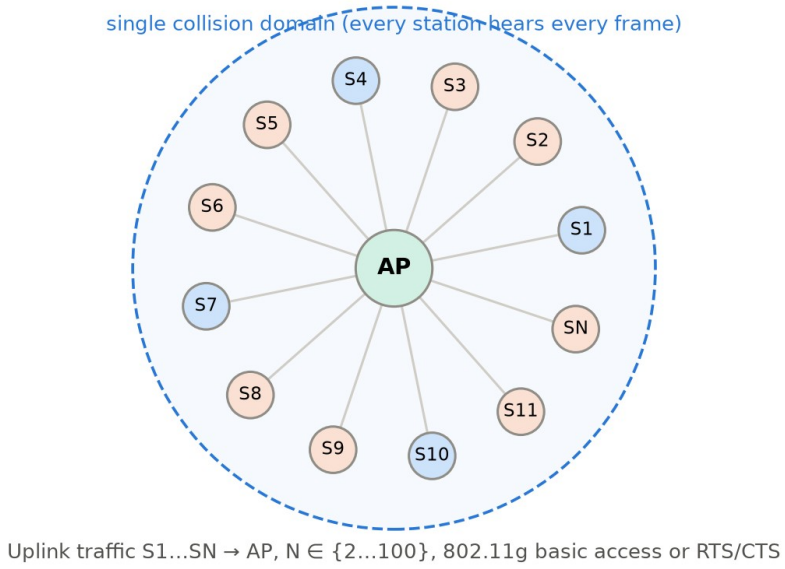


Figure 3.1: Simulated topology: a single 802.11g BSS with N stations in one collision domain.

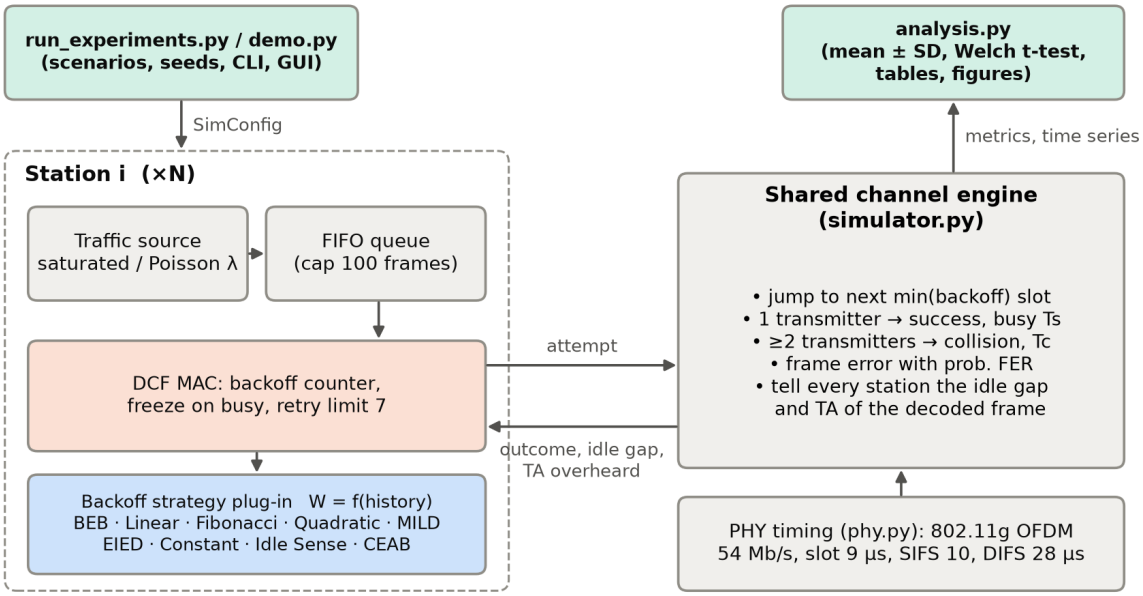


Figure 3.2: Architecture of the simulator and evaluation framework.

3.2 Baseline mechanism description

The baseline is the standard 802.11 DCF with binary exponential backoff (left half of Figure 3.3). With W0 = CWmin+1 = 16 and m = 6 doubling stages the window takes the values 16, 32, …, 1024. After a success, or after a frame is dropped once the retry limit of 7 is exceeded, W returns to 16. BEB therefore adapts to collisions only within the lifetime of one frame and forgets the contention level between frames. The other seven reference strategies implemented for comparison change only the update rule:

Table 3.1: Update rules of the reference waiting strategies (W0 = 16, Wmax = 1024).

Strategy	On failure	On success	Source
BEB (baseline)	$W \leftarrow \min(2W, W_{max})$	$W \leftarrow W_0$	IEEE 802.11 [1]
Constant	W unchanged (64)	W unchanged	p-persistent CSMA [11]
Linear	$W \leftarrow W + W_0$	$W \leftarrow W_0$	—
Fibonacci	$W \leftarrow W_0 \cdot F(\text{retry})$, F = 1, 2, 3, 5, 8, …	$W \leftarrow W_0$	[14]
Quadratic	$W \leftarrow W_0 \cdot (\text{retry}+1)^2$	$W \leftarrow W_0$	polynomial backoff [15]
MILD	$W \leftarrow 1.5\,W$	$W \leftarrow W - 1$	MACAW [4]
EIED	$W \leftarrow 2\,W$	$W \leftarrow W / 2$	[5]
Idle Sense	— (no per-collision change)	AIMD on W every 5 busy periods towards the optimal mean idle-slot count	[6]

3.3 Proposed mechanism: algorithm, pseudocode, flowchart

Contender-Estimation Adaptive Backoff (CEAB) replaces BEB's 'double and reset' rule with a window derived from an estimate of how many stations are actually contending. It has four parts.

(a) Contender estimation from overheard addresses

Every station records the transmitter address of each successfully decoded data frame on the channel (its own included) and keeps the last M = 200 of them. If cj frames in this window came from station j, the effective number of contenders is the unbiased inverse Simpson index

$$n_eff = M(M - 1) / \Sigma j\ cj(cj - 1)$$

When n stations share the channel equally, n_eff = n. Stations that send rarely contribute little, so n_eff measures how many stations are *competing*, not how many are associated. The estimate uses no model of the other stations' backoff rule, and because all stations hear the same frames, they all compute the same value.

(b) Load factor for unsaturated traffic

A station that sends one frame per second still appears in the address window, but it is rarely backlogged. CEAB therefore keeps an exponentially weighted average I_obs (weight 0.05) of the idle time between busy periods and compares it with I*, the mean idle gap at the analytical throughput optimum (≈ 4.0 slots for 802.11g):

$$\rho = min(1, I^* / I_obs), \quad n_c = max(1, \rho \cdot n_eff)$$

In saturation the channel is idle about as long as at the optimum, so ρ ≈ 1. When traffic is light the channel is idle much longer, ρ ≪ 1, and the window shrinks towards CWmin. The loop corrects itself: a window that is too large produces extra idle time, which lowers ρ and shrinks the window.

(c) Optimal target window

For the PHY timing in use (slot σ, success time Ts and collision time Tc), the per-slot attempt probability τ*(n) that maximises Bianchi's saturation throughput S(τ, n) is pre-computed by numerical optimisation for n = 1 … 1000. Since a window W gives τ = 2/(W+1), the target window is

$$W = 2 / \tau(n_c) - 1$$

The base window moves halfway towards this target after each of the station's own outcomes: W_base ← W_base + γ(W* − W_base), γ = 0.5, clamped to [CWmin+1, CWmax+1], so the scheme is never more aggressive than the standard minimum.

(d) Window used for each attempt

On success the next frame starts with W = W_base — never back at CWmin. On failure the station may escalate within one frame, W = W_base · 2^min(retry, K). The ablation study (Section 5.4) shows that K = 0, meaning no escalation at all, gives the lowest tail delay, so K = 0 is the default: collisions do not change the window, only the estimate does. Figure 3.3 shows the BEB and CEAB flowcharts side by side.

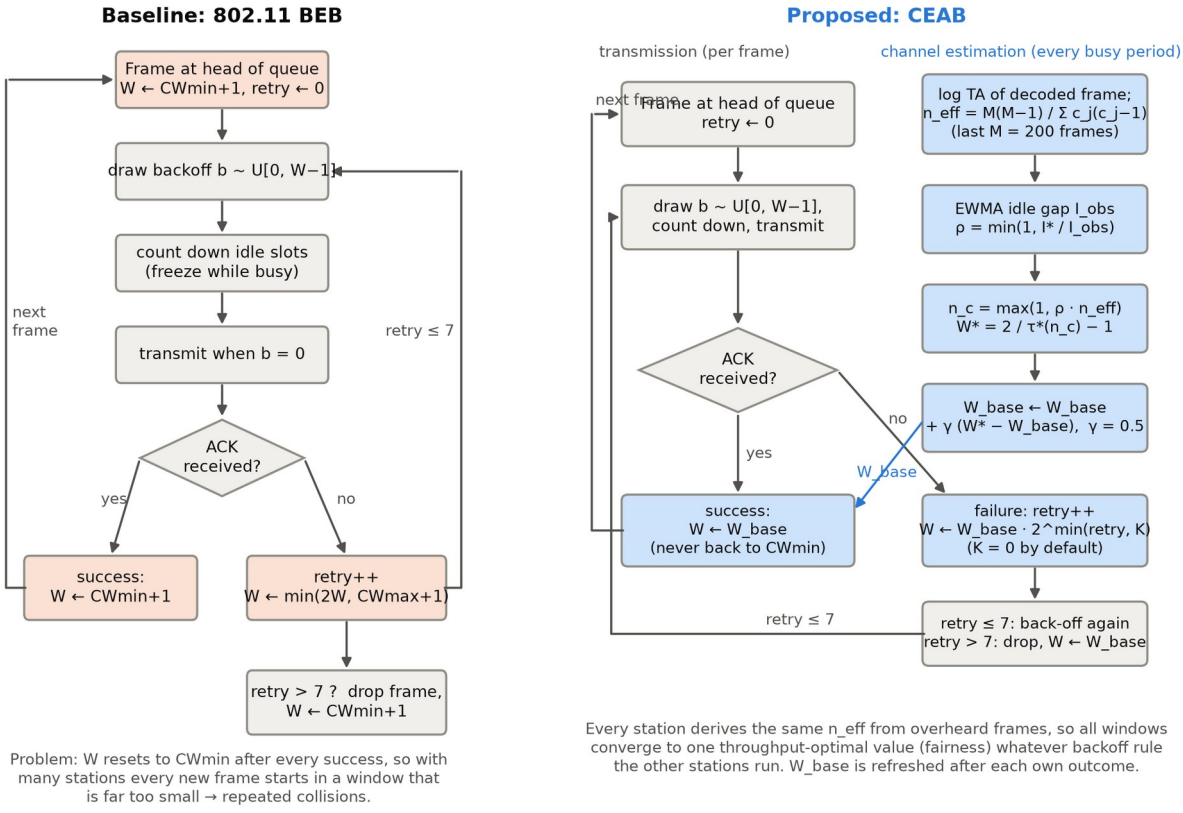


Figure 3.3: Flowcharts of the baseline (802.11 BEB) and the proposed CEAB.

Pseudocode

```
Algorithm 1  CEAB (executed independently by every station i)
Parameters: M = 200, gamma = 0.5, beta = 0.05, K = 0, W0 = CWmin+1, Wmax = CWmax+1
State:      log (TA of recently decoded frames), I_obs, W_base = W0, W = W0
Precompute: tau_star[n] = argmax_tau S(tau, n; Ts, Tc, slot)   for n = 1..1000
            I_star      = optimal mean idle gap (slots) * slot

on every busy period (idle gap g_us, decoded frame from station s or none):
    if a frame was decoded:  append s to log
```

```
    I_obs <- (1 - beta) * I_obs + beta * g_us

function target_window():
    win   <- last min(M, |log since join|) entries of log
    c_j   <- count of station j in win;  m <- |win|
    n_eff <- m(m-1) / sum_j c_j(c_j-1)          (n_eff = m if the sum is 0)
    rho   <- min(1, I_star / I_obs)
    n_c   <- max(1, rho * n_eff)
    return 2 / tau_star[n_c] - 1

on own transmission outcome (success or failure):
    W_base <- clip(W_base + gamma * (target_window() - W_base), W0, Wmax)
    if success:           W <- W_base;  retry <- 0
    else:                 retry <- retry + 1
                          if retry > 7: drop frame; W <- W_base; retry <- 0
                          else:         W <- min(W_base * 2^min(retry, K), Wmax)
    draw backoff b ~ Uniform{0, ..., round(W) - 1}
```

3.4 Novelty statement

Compared with the 802.11 BEB baseline, CEAB differs in exactly the following ways:

- **What sets the window:** BEB uses only the retry count of the current frame; CEAB uses an estimate of the number of contending stations taken from the channel.
- **Behaviour after success:** BEB resets W to CWmin (16 slots); CEAB keeps W at the estimated optimum (≈ 430 slots with 50 stations, ≈ 40 with 5), which removes the post-success collision burst and the capture effect.
- **Behaviour after a collision:** BEB doubles W; CEAB (K = 0) leaves W unchanged, because one collision says little about the number of stations.
- **Estimator:** unlike Calì et al. [3] and Bianchi & Tinnirello [7], the estimate is model-free (inverse Simpson index of overheard transmitter addresses) and stays valid whatever backoff rule the other stations use; unlike Idle Sense [6], the window is not steered towards a fixed idle-slot target, so CEAB stations are not starved next to legacy stations.
- **Unsaturated traffic:** the idle-gap load factor ρ corrects the estimate when stations are associated but not backlogged; none of the history-based rules distinguishes the two cases.

3.5 Assumptions and design decisions

- Single collision domain: every station decodes every frame (no hidden terminals), so all stations see the same address sequence and idle gaps.
- Frame errors are independent with probability FER and, like collisions, are detected only by a missing ACK.
- The CEAB window never goes below the standard CWmin+1 = 16, so CEAB is never more aggressive than a legacy station (a deliberate coexistence choice).

- M = 200 frames is about 70 ms of airtime at 54 Mb/s: long enough to see each of 50 saturated stations about four times, short enough to follow churn. The ablation shows results are insensitive to M between 50 and 1000.
- τ*(n) is computed from the analytical model rather than learned online. The model is accurate near the optimum (Figure 5.2), and a check by simulation confirmed that the throughput plateau around W* is flat.
- Post-backoff after an idle period and EIFS after a corrupted frame are simplified: a newly arrived frame always draws a backoff, and a corrupted frame costs Tc.

4. IMPLEMENTATION

4.1 Tools, platform, versions and hardware

Table 4.1: Implementation environment.

Item	Details
Language	Python 3.14.4
Libraries	NumPy 2.5.4, SciPy 1.18.1, Matplotlib 3.11.2 (Tk backend for the GUI), python-docx / python-pptx for documents
Simulator	Custom event-driven 802.11 DCF simulator (package csmasim), written for this project
Operating system	Linux 7.0.3-arch1-2 (Arch Linux)
Hardware	16-core x86-64 workstation; experiments run in parallel with multiprocessing
Run time	2170 independent simulations (10 simulated seconds each) in about 6 minutes
Reproducibility	Every run is seeded (seeds 1…10); results/raw_runs.csv stores each run

4.2 Simulation setup

The simulated network is the BSS of Figure 3.1. The PHY/MAC parameters follow 802.11g ERP-OFDM; the data frame and ACK durations are computed with the OFDM symbol rule (20 µs preamble + 4 µs per symbol). Table 4.2 lists all parameters.

Table 4.2: Simulation parameters.

Parameter	Value
PHY	IEEE 802.11g ERP-OFDM, data rate 54 Mb/s, control rate 24 Mb/s
Slot time σ / SIFS / DIFS	9 µs / 10 µs / 28 µs
Payload / MAC header + FCS	1500 B (256–2304 B in the payload study) / 28 B
Data frame / ACK duration	248 µs / 28 µs
Ts / Tc (basic access)	314 µs / 323 µs
Ts / Tc (RTS/CTS)	390 µs / 103 µs
CWmin / CWmax / retry limit	15 / 1023 / 7 (window sizes 16 and 1024)
Number of stations N	2, 5, 10, 20, 35, 50, 75, 100
Traffic	Saturated, or Poisson with 2–40 Mb/s total offered load (N = 30), queue 100 frames
Frame error rate	0, 0.05, 0.1, 0.2, 0.3
Simulated time / warm-up	10 s per run / first 1 s discarded
Independent runs	10 per configuration (seeds 1–10); churn: 5 runs
CEAB parameters	M = 200, γ = 0.5, β = 0.05, K = 0, W floor = 16
Idle Sense parameters	ε = 6, α = 1/1.0666, maxtrans = 5, target from the analytical optimum

4.3 Code structure and explanation of key modules

Table 4.3: Source files.

File	Lines	Responsibility
csmasim/phy.py	67	802.11g timing: frame durations, Ts, Tc, DIFS; basic and RTS/CTS
csmasim/analytical.py	75	Bianchi fixed point; throughput S(τ, n); table of optimal τ*(n)
csmasim/strategies.py	270	Strategy base class and the nine strategies, including CEAB
csmasim/simulator.py	348	Event-driven DCF engine, traffic, churn, metrics, event trace
csmasim/experiments.py	132	Scenario definitions and parallel execution of all runs

File	Lines	Responsibility
csmasim/analysis.py	391	Mean ± SD, Welch t-tests, tables and figures
run_experiments.py	45	Command-line entry point for the full evaluation
demo.py	253	Live demo: compare, trace and GUI with sliders
tests/test_simulator.py	94	Nine unit/validation tests

Simulation engine (simulator.py). Instead of stepping through every 9 µs slot, the engine jumps directly to the next slot in which some backoff counter reaches zero (or to the next packet arrival or churn event), subtracting the skipped idle slots from all counters at once with NumPy. If exactly one station's counter reaches zero and no frame error is drawn, the slot becomes a success of length Ts; otherwise it is a collision or corrupted frame of length Tc. All counters are frozen during the busy period, as in the standard. After the busy period the engine calls observe() on strategies that listen to the channel, then on_success, on_failure or on_drop on the transmitters, which draw new backoff values.

Strategy interface (strategies.py). A strategy object manages the windows of a group of stations as a NumPy array and implements window(), on_success(), on_failure(retry), on_drop(), observe(...) and reset(). A network can mix groups — for example 10 BEB and 10 CEAB stations — which is how the coexistence experiment is built. CEAB keeps the shared address log, the per-station join position (so that a newly joined station uses only frames it has heard itself), the idle-gap average and W_base.

Analytical module (analytical.py). It solves Bianchi's fixed-point equations with Brent's method for validation, and tabulates $\tau^*(n)$ by bounded scalar optimisation of $S(\tau, n)$ for the current Ts, Tc and slot; the table is cached per PHY configuration.

Experiments and analysis. experiments.py expands the scenario list into 2170 (scenario, parameter, strategy, seed) jobs, runs them on all CPU cores and writes one CSV row per run. analysis.py computes mean and sample standard deviation, Welch's t-test between BEB and CEAB, the result tables and every figure in Chapter 5.

4.4 Challenges faced and how they were solved

Table 4.4: Challenges and solutions.

Challenge	Solution
A slot-by-slot loop in Python was far too slow for 100 stations × 10 s × 2000+ runs.	Event skipping: jump to min(counters) and decrement all counters in one vectorised step; run jobs in parallel. All 2170 runs finish in about 6 minutes.
Is the simulator correct?	Validated against Bianchi's model (throughput error ≤ 1.7 %, collision probability within 0.023) and against an independent naive slot-by-slot Monte Carlo (unit tests).

Challenge	Solution
First CEAB design estimated n from the station's own attempt rate. Stations with a larger window measured a lower attempt rate, estimated more contenders and enlarged their window further — a positive feedback loop (short-term fairness only 0.43).	The estimate must come from the channel only, so that every station computes the same value.
Second design inverted the analytical model from the observed idle and success probabilities. Measured idle gaps were about 0.8 slot longer than the i.i.d. model predicts (backoff counters are correlated), and the inversion is ill-conditioned, so estimates exploded to 10^4 stations.	Replaced by a model-free estimator: counting transmitter addresses (inverse Simpson index). The estimate is within a few per cent of the true N (Figure 5.7).
Under light Poisson traffic CEAB had twice the delay of BEB: idle periods with no contender were not counted, so the channel looked busier than it was.	The load factor now uses the wall-clock idle gap and compares it with the optimal gap; low-load delay now equals BEB's.
MILD runs at N = 5–10 had very high variance.	Not a bug: individual runs lock in at CWmax because a −1 decrease cannot undo ×1.5 increases. Reported as a finding (Section 5.4).

5. EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

Table 5.1: Performance metrics.

Metric	Definition	Why it matters
Throughput (Mb/s)	Payload bits delivered per second (after warm-up)	Primary capacity measure; the quantity the optimal window maximises
Collision probability	Failed-by-collision attempts / all attempts	Direct indicator of a window that is too small; wastes airtime and energy
Mean access delay (ms)	Time from a frame reaching the head of the queue to its ACK	MAC service time; with saturation it equals N / frame rate
99th-percentile access delay (ms)	Tail of the access-delay distribution	Real-time and interactive traffic suffers from the tail, not the mean
Short-term Jain fairness	$J = (\Sigma x)^2 / (N \cdot \Sigma x^2)$ of per-station frame counts in 100 ms windows, averaged	Captures the capture effect that long-term averages hide
Drop rate	Frames discarded after 7 retries / frames handled	Losses visible to upper layers
End-to-end delay (ms)	Arrival in the queue to ACK (Poisson traffic)	Includes queueing; shows the saturation point

5.2 Scenarios and parameter values

Table 5.2: Evaluation scenarios (2170 simulation runs in total).

Scenario	Configuration	Strategies	Runs
S1 / S2 / S3	Saturated, N = 5 / 20 / 50 (low / medium / high contention)	all 9	10
Station sweep	Saturated, N = 2 … 100	all 9	10
Validation	BEB with unlimited retries vs. Bianchi model, N = 2 … 100	BEB	10
Offered load	Poisson, N = 30, 2 … 40 Mb/s total	BEB, EIED, Idle Sense, CEAB	10
Channel errors	Saturated, N = 20, FER 0 … 0.3	4 main	10
RTS/CTS	Saturated, N = 5 / 20 / 50, RTS/CTS access	4 main	10
Payload	Saturated, N = 20, 256 … 2304 B	4 main	10
Churn	N: 5 → 50 at t = 4 s → 15 at t = 8 s, 12 s runs	4 main	5
Coexistence	10 legacy BEB + 10 stations of scheme X	X = BEB, EIED, Idle Sense, CEAB	10
Ablation	CEAB with $K \in \{0,1,2\}$, $M \in \{50 \ldots 1000\}$, N = 20, 50	CEAB	10

5.3 Results: baseline vs proposed

Table 5.3 follows the results template: BEB (baseline) against CEAB (proposed) for the low, medium and high load scenarios, as the mean ± standard deviation of 10 independent runs. Improvement is positive when CEAB is better (higher throughput or fairness, lower collision probability, delay or drop rate). The last column gives the p-value of Welch's two-sample t-test.

Table 5.3: Baseline (BEB) vs proposed (CEAB) — mean ± SD over repeated runs.

Scenario	Metric	Baseline (mean ± SD)	Proposed (mean ± SD)	Improve-ment (%)	p-value
S1 (low load, N=5)	Throughput (Mb/s)	30.05 ± 0.07	30.61 ± 0.08	+1.9	2.6e-12
	Collision probability	0.265 ± 0.003	0.187 ± 0.004	+29.2	7.8e-20
	Mean access delay (ms)	1.99 ± 0.01	1.96 ± 0.01	+1.4	6.0e-06
	99th-pct access delay (ms)	15.39 ± 0.43	5.99 ± 0.07	+61.1	4.5e-14
	Short-term Jain fairness (100 ms)	0.917 ± 0.004	0.993 ± 0.000	+8.2	5.2e-13
	Frame drop rate	0.005% ± 0.004%	0.000% ± 0.000%	+100.0	6.7e-03
S2 (medium load, N=20)	Throughput (Mb/s)	25.84 ± 0.06	29.93 ± 0.10	+15.8	8.0e-22
	Collision probability	0.468 ± 0.002	0.207 ± 0.005	+55.7	2.1e-21
	Mean access delay (ms)	8.34 ± 0.09	8.02 ± 0.03	+3.9	2.7e-07
	99th-pct access delay (ms)	127.05 ± 2.74	25.63 ± 0.27	+79.8	7.3e-16
	Short-term Jain fairness (100 ms)	0.599 ± 0.008	0.965 ± 0.001	+61.1	2.1e-16
	Frame drop rate	0.345% ± 0.033%	0.000% ± 0.000%	+100.0	1.1e-10
S3 (high load, N=50)	Throughput (Mb/s)	22.33 ± 0.06	29.81 ± 0.07	+33.5	6.7e-32
	Collision probability	0.595 ± 0.002	0.204 ± 0.004	+65.7	3.4e-27
	Mean access delay (ms)	20.00 ± 0.33	20.13 ± 0.05	-0.6	2.8e-01
	99th-pct access delay (ms)	304.04 ± 5.51	64.74 ± 0.89	+78.7	6.9e-17
	Short-term Jain fairness (100 ms)	0.397 ± 0.006	0.910 ± 0.002	+129.1	9.9e-22
	Frame drop rate	2.007% ± 0.107%	0.000% ± 0.000%	+100.0	5.4e-13

Table 5.4: Throughput (Mb/s) of every strategy in S1–S3 (mean ± SD).

Strategy	S1 (N = 5)	S2 (N = 20)	S3 (N = 50)
BEB (802.11)	30.05 ± 0.07	25.84 ± 0.06	22.33 ± 0.06
Constant W=64	30.06 ± 0.05	26.22 ± 0.07	15.65 ± 0.12
Linear	29.55 ± 0.07	21.97 ± 0.06	14.40 ± 0.05
Fibonacci	29.69 ± 0.08	23.99 ± 0.08	19.09 ± 0.07
Quadratic	30.59 ± 0.06	27.20 ± 0.05	23.64 ± 0.05
MILD	29.34 ± 6.53	21.72 ± 0.06	27.91 ± 0.05
EIED	30.50 ± 0.07	28.44 ± 0.05	26.79 ± 0.05
Idle Sense	30.57 ± 0.07	29.67 ± 0.08	29.06 ± 0.10
CEAB (proposed)	**30.61 ± 0.08**	**29.93 ± 0.10**	**29.81 ± 0.07**

Table 5.5: 99th-percentile access delay (ms) in S1–S3 (mean ± SD).

Strategy	S1 (N = 5)	S2 (N = 20)	S3 (N = 50)
BEB (802.11)	15.39 ± 0.43	127.05 ± 2.74	304.04 ± 5.51
Constant W=64	5.75 ± 0.06	33.65 ± 0.22	82.47 ± 0.49
Linear	12.39 ± 0.26	64.08 ± 0.68	100.89 ± 0.85
Fibonacci	13.17 ± 0.22	98.28 ± 5.27	190.57 ± 3.97
Quadratic	17.88 ± 0.23	108.22 ± 3.04	265.36 ± 5.20
MILD	42.67 ± 10.65	27.33 ± 0.71	63.34 ± 0.43
EIED	16.35 ± 0.53	138.24 ± 3.84	292.89 ± 7.47
Idle Sense	5.90 ± 0.08	26.90 ± 0.43	70.99 ± 1.10
CEAB (proposed)	**5.99 ± 0.07**	**25.63 ± 0.27**	**64.74 ± 0.89**

Table 5.6: Short-term Jain fairness index in S1–S3 (mean ± SD).

Strategy	S1 (N = 5)	S2 (N = 20)	S3 (N = 50)
BEB (802.11)	0.917 ± 0.004	0.599 ± 0.008	0.397 ± 0.006
Constant W=64	0.993 ± 0.001	0.946 ± 0.001	0.751 ± 0.006
Linear	0.970 ± 0.003	0.806 ± 0.005	0.598 ± 0.006
Fibonacci	0.957 ± 0.007	0.690 ± 0.007	0.481 ± 0.004
Quadratic	0.923 ± 0.008	0.673 ± 0.008	0.475 ± 0.005
MILD	0.295 ± 0.231	0.959 ± 0.002	0.915 ± 0.002

Strategy	S1 (N = 5)	S2 (N = 20)	S3 (N = 50)
EIED	0.832 ± 0.016	0.425 ± 0.007	0.274 ± 0.004
Idle Sense	0.993 ± 0.001	0.963 ± 0.002	0.899 ± 0.002
CEAB (proposed)	**0.993 ± 0.000**	**0.965 ± 0.001**	**0.910 ± 0.002**

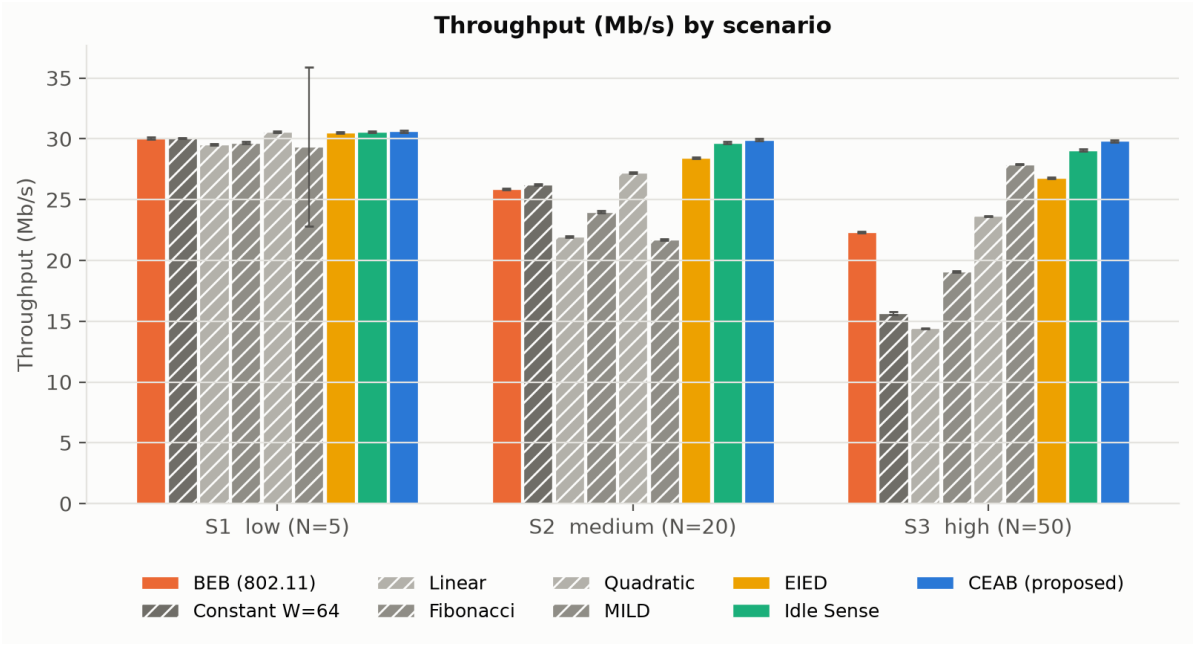


Figure 5.1: Throughput of all strategies in scenarios S1–S3 (whiskers = ±1 SD).

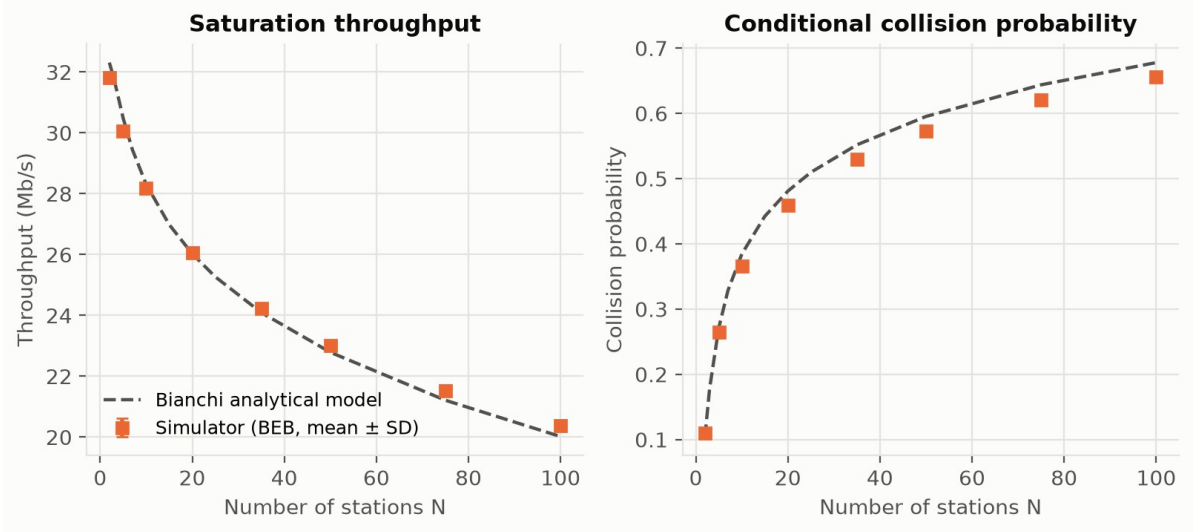


Figure 5.2: Validation of the simulator against the Bianchi analytical model.

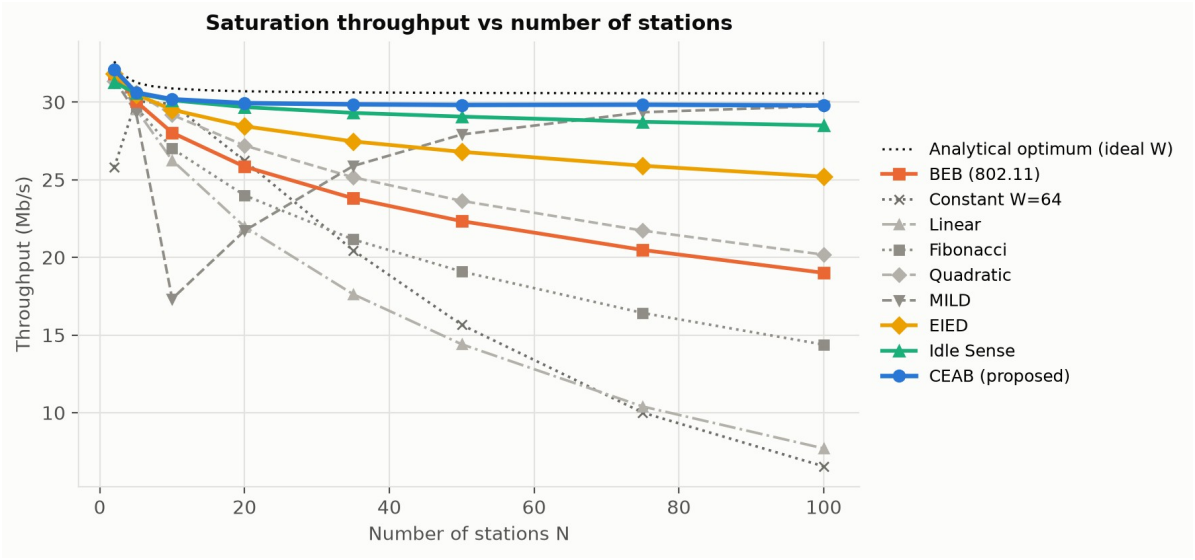


Figure 5.3: Saturation throughput as the number of stations grows from 2 to 100.

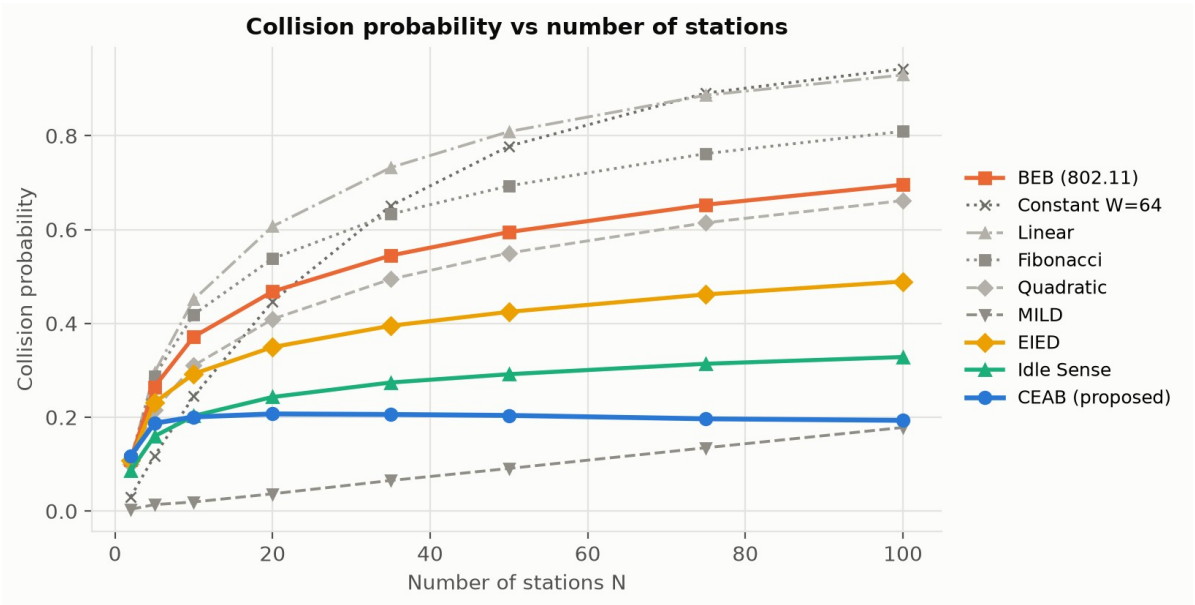


Figure 5.4: Collision probability versus number of stations.

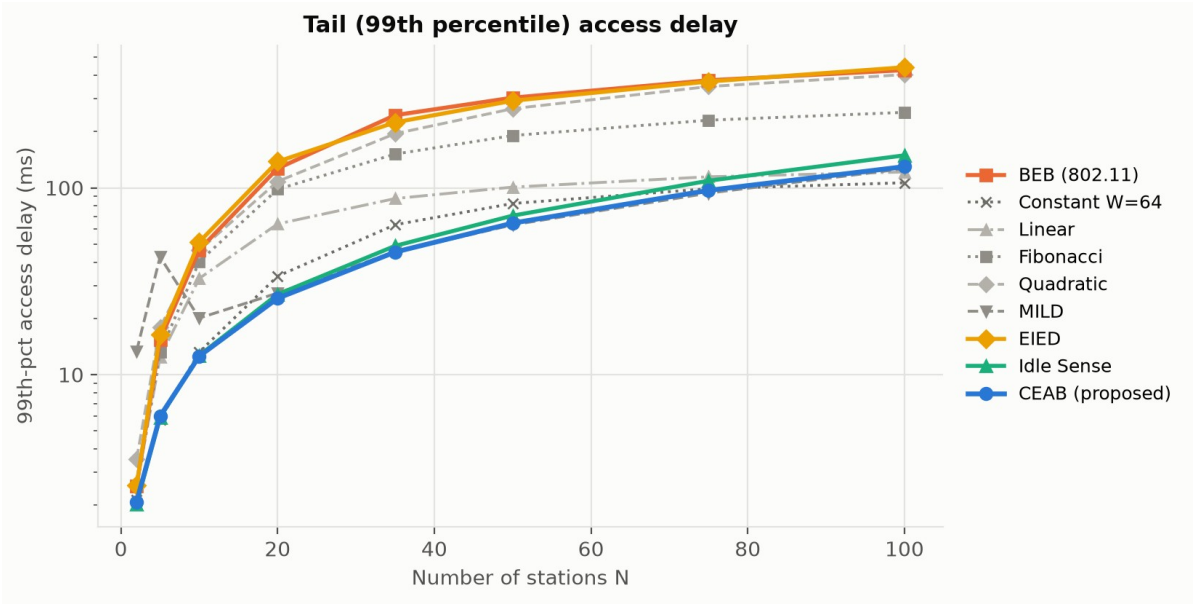


Figure 5.5: Tail (99th percentile) access delay versus number of stations (log scale).

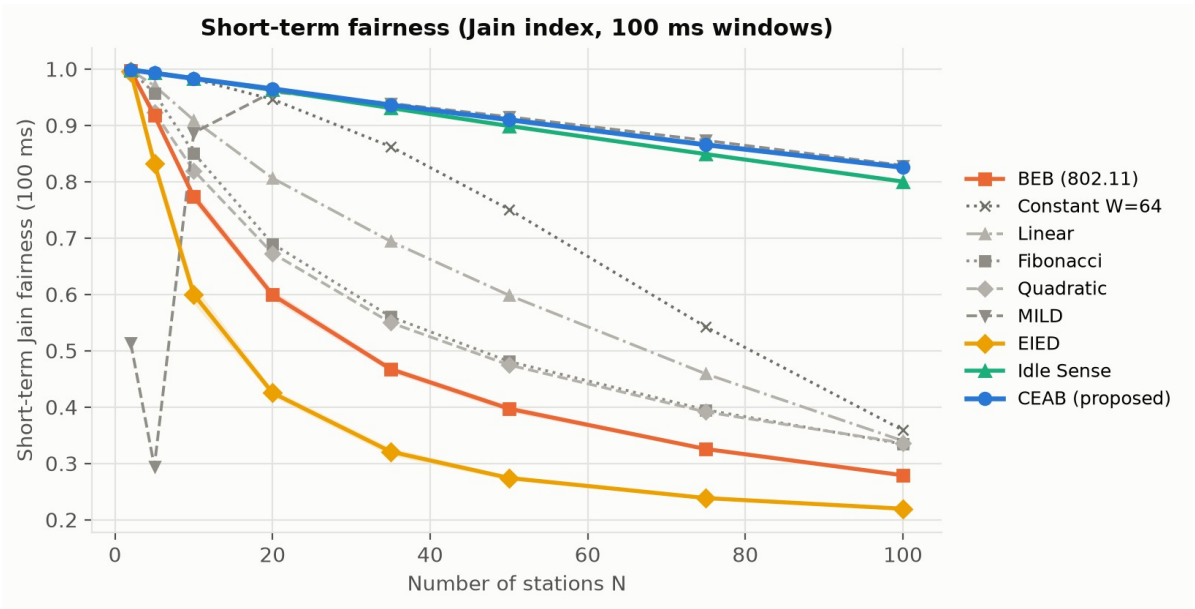


Figure 5.6: Short-term (100 ms) Jain fairness versus number of stations.

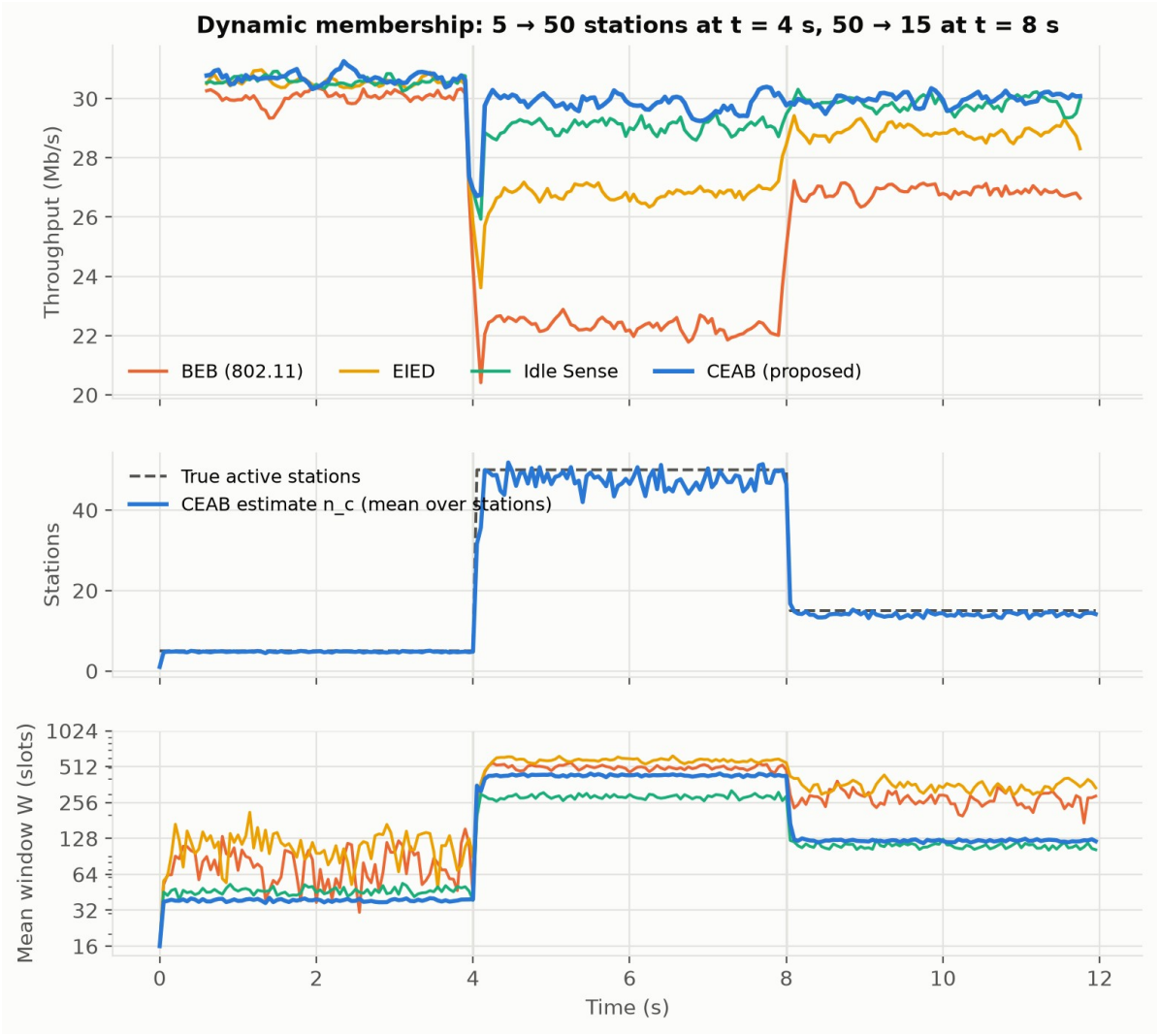


Figure 5.7: Dynamic membership: throughput, CEAB's estimate and mean window over time.

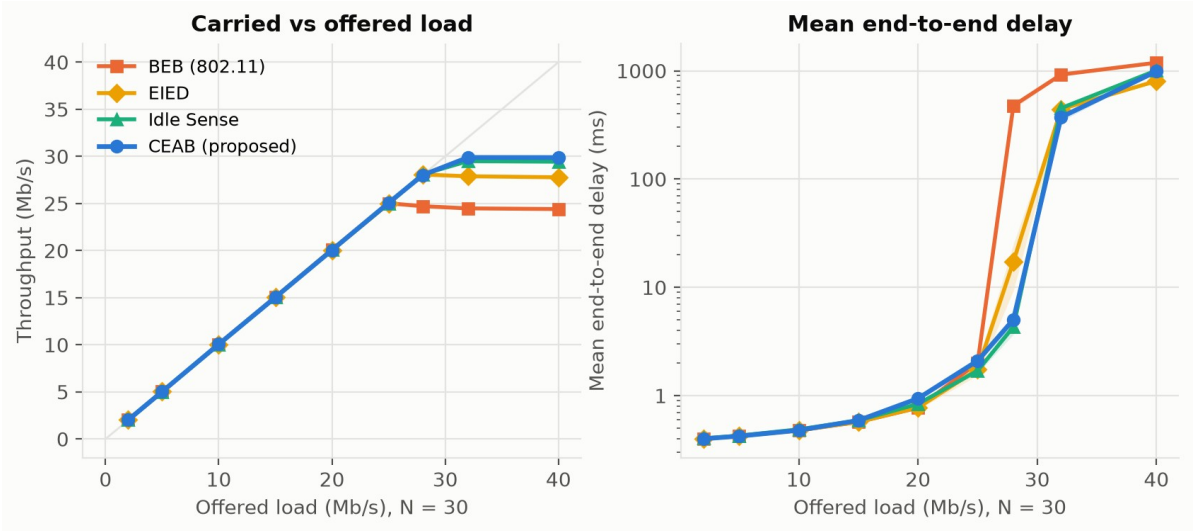


Figure 5.8: Unsaturated Poisson traffic with 30 stations: carried load and end-to-end delay.

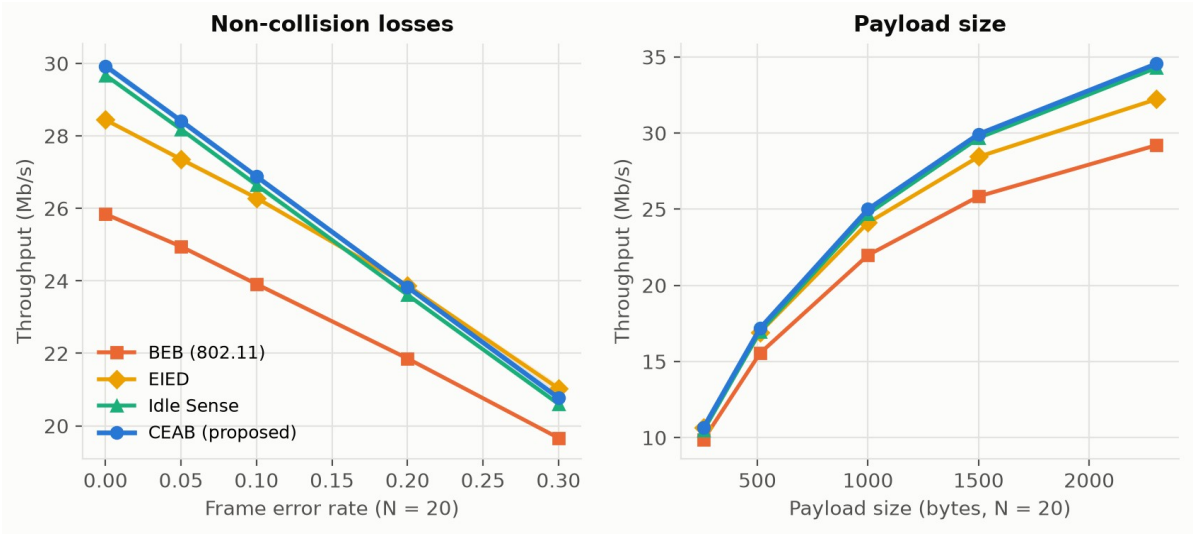


Figure 5.9: Sensitivity to frame errors (left) and payload size (right), N = 20.

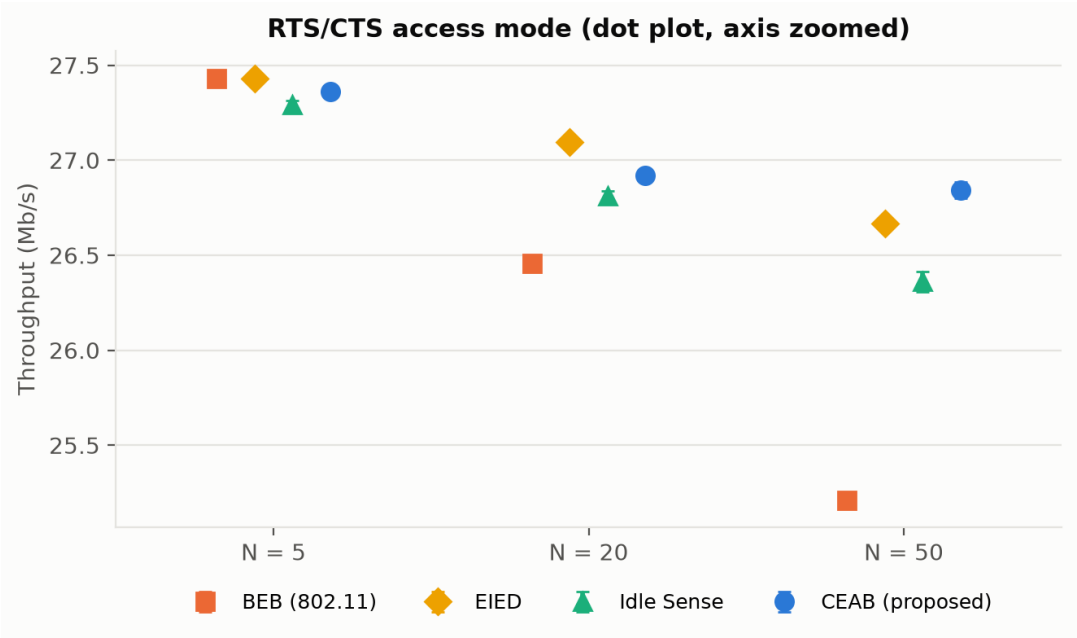


Figure 5.10: Throughput with RTS/CTS access.

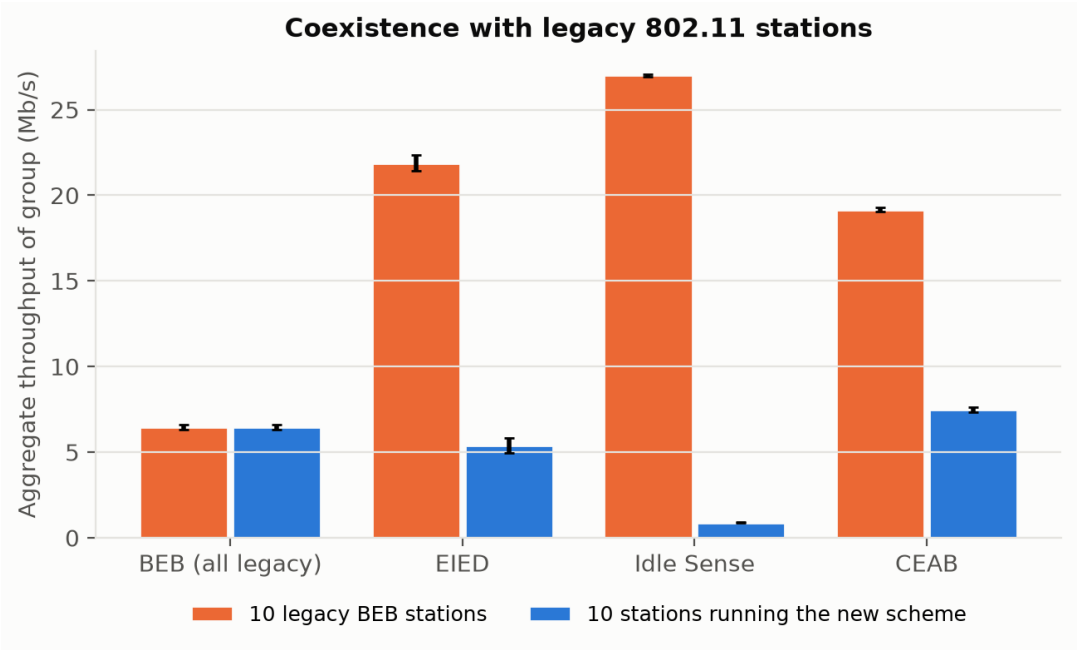


Figure 5.11: Coexistence of 10 legacy BEB stations with 10 stations running each scheme.

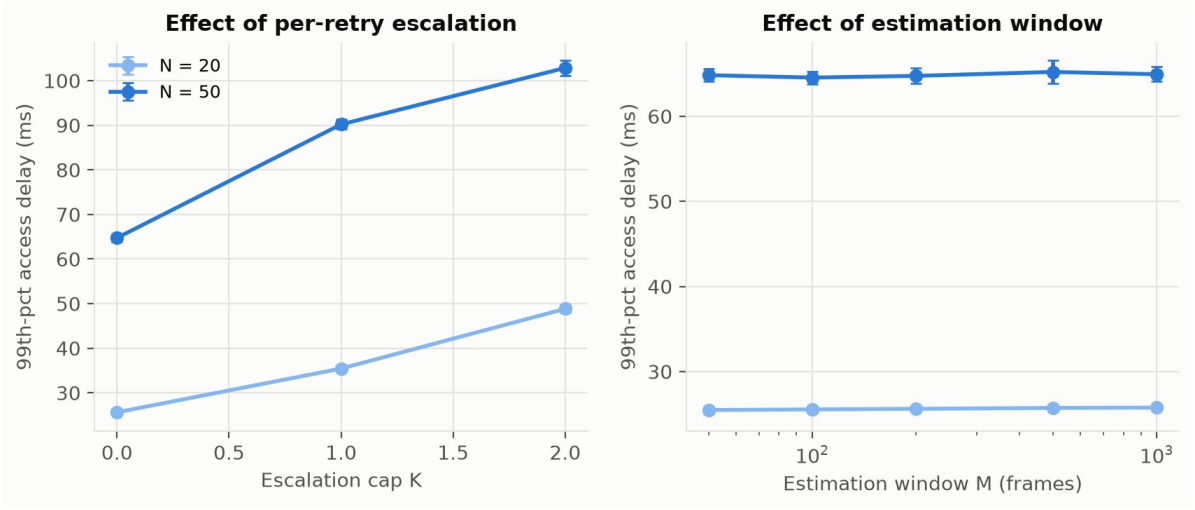


Figure 5.12: Ablation: escalation cap K and estimation window M of CEAB.

5.4 Analysis and discussion

Validation

Before any comparison, the simulator was checked against Bianchi's model. Figure 5.2 shows that simulated BEB throughput and collision probability follow the analytical curves for 2 to 100 stations, with a maximum throughput error of 1.7 % and a maximum collision-probability difference of 0.023. The small remaining gap at large N comes from the model's assumption that collision probability is independent of the backoff stage.

Why BEB loses capacity as N grows

BEB throughput falls from 31.8 Mb/s with 2 stations to 22.3 Mb/s with 50 and 19.0 Mb/s with 100, while the theoretical limit stays near 30.6 Mb/s (Figure 5.3). The reason is visible in the churn trace (Figure 5.7, bottom): BEB's mean window with 50 stations is large, but every station that has just succeeded is back at 16, so the next attempts collide in bursts. The collision probability reaches 0.59 (Figure 5.4) and 2 % of the frames are dropped after 7 retries.

How CEAB recovers it

CEAB keeps every station at the window that suits the estimated contender count. Its throughput is 30.61, 29.93 and 29.81 Mb/s in S1–S3 — +1.9 %, +15.8 % and +33.5 % over BEB — and 29.79 Mb/s at N = 100 (+57 %). Across the whole sweep it stays at 97.5 % or more of the analytical optimum. The collision probability is almost independent of N (about 0.20), which is the signature of operating at the optimal τ. Every difference in Table 5.3 except the mean access delay at N = 50 is significant at $p < 0.01$. With saturated stations the mean access delay is fixed by the frame rate (each of N stations gets 1/N of the successes), so the mean barely changes; the gain appears in the tail: the 99th-percentile delay falls from 304 ms to 65 ms at N = 50.

Fairness and the capture effect

Short-term fairness is where the waiting strategies differ most (Figure 5.6). BEB drops to 0.40 with 50 stations and EIED to 0.27, because the station that just succeeded has the smallest window (BEB) or keeps a smaller window than its neighbours (EIED). CEAB stations all derive the same estimate from the same overheard frames and therefore use the same window, giving 0.91. Idle Sense, which also equalises windows, reaches 0.90.

Comparison of the waiting strategies

Growth rate alone does not fix the problem. Linear and Fibonacci backoff grow more slowly than BEB and collide even more (0.81 and 0.69 at N = 50). Quadratic growth is a little better than BEB, and a fixed window of 64 is excellent for 5–10 stations but collapses to 15.7 Mb/s at N = 50 because it cannot adapt. The schemes that do not reset after a success — EIED, MILD, Idle Sense and CEAB — are the only ones that hold up at high N, which supports the central idea of this project: the window must carry information about contention from one frame to the next. MILD illustrates the danger of doing so badly. It is close to CEAB at N =

75–100 (29.7 Mb/s), but at N = 5 its throughput has a standard deviation of 6.5 Mb/s: in some runs a few stations drift to CWmax and stay there, because one ×1.5 increase needs dozens of −1 decreases to undo. Idle Sense is the strongest reference scheme; CEAB beats it slightly in throughput (by 2.6 % at N = 50 and 4.5 % at N = 100), in collision probability and in tail delay, and matches its fairness.

Dynamic membership

When 45 stations join at t = 4 s (Figure 5.7), CEAB's estimate reaches within 15 % of the new count in about 150 ms, and after the drop to 15 stations at t = 8 s in about 50 ms. Between 4.5 s and 8 s CEAB delivers 29.8 Mb/s against 22.3 Mb/s for BEB and 29.1 Mb/s for Idle Sense, whose additive window increase needs longer to reach the larger window.

Unsaturated traffic

With Poisson traffic (Figure 5.8) all schemes carry the offered load up to about 25 Mb/s and have the same delay at light load (≈ 0.42 ms at 5 Mb/s), which confirms that the load factor shrinks CEAB's window when stations are rarely backlogged. BEB saturates at about 24.4 Mb/s, while CEAB carries up to 29.9 Mb/s (+22 %). At 28 Mb/s offered, BEB is already overloaded (mean delay 476 ms) while CEAB still delivers the full load with 5.0 ms.

Sensitivity: frame errors, payload and RTS/CTS

Random frame errors (Figure 5.9, left) hurt every scheme, but BEB also misreads each error as a collision and doubles its window. CEAB's estimate is unaffected (it counts decoded frames and idle time), so it stays ahead of BEB at every error rate. The gain narrows, however, from 15.8 % at FER = 0 to 5.6 % at FER = 0.3, where EIED is slightly better. The gain grows with payload size (8.0 % at 256 B, 18.3 % at 2304 B), because larger frames make collisions more expensive. With RTS/CTS a collision costs only 103 µs instead of 323 µs, so the optimal window is smaller and BEB is already close to it: CEAB's throughput gain falls to -0.3 %, +1.7 % and +6.5 % for N = 5, 20 and 50. Its tail-delay and fairness advantages remain (p99 271 → 80 ms at N = 50).

Coexistence with legacy stations

When 10 CEAB stations share the channel with 10 legacy BEB stations (Figure 5.11), the total throughput rises from 13.0 Mb/s (all BEB) to 26.6 Mb/s. The split is uneven, though: the BEB group gets 19.2 Mb/s and the CEAB group 7.5 Mb/s, because legacy stations restart at

W = 16 after every success. Even so, a CEAB station gets 0.75 Mb/s, more than the 0.65 Mb/s it would get in an all-BEB cell, so adopting CEAB never makes a station worse off. Idle Sense fares much worse in the same test (0.89 Mb/s for its 10 stations), because the BEB stations keep the idle time below its target, so it keeps enlarging its window.

Ablation

Figure 5.12 isolates CEAB's design parameters. Allowing per-retry doubling on top of the estimate (K = 1 or 2) does not improve throughput but raises the N = 50 tail delay from 65 ms to 90 and 103 ms and lowers fairness. In a single collision domain the estimate is accurate, so reacting to individual collisions only adds noise; hence K = 0. The estimation window M has almost no effect between 50 and 1000 frames (p99 64.5–65.2 ms), so it can be chosen for tracking speed (M = 200).

5.5 Critical evaluation

Strengths

- Close to the theoretical capacity for every N from 5 to 100 (≥ 97.5 % of the optimum); the gain over BEB grows with N.
- Removes the capture effect: short-term fairness of 0.91 or more up to 50 stations.
- Tail delay reduced by 79 % at N = 50, and no frames dropped at the retry limit.
- Uses only information every 802.11 receiver already decodes; no new frames, fields or AP support.
- The estimator does not depend on the other stations' backoff rule, so the scheme degrades gracefully in mixed networks (unlike Idle Sense).
- Only one parameter matters in practice (K), and the ablation shows a clear best value.

Weaknesses

- At medium unsaturated load the mean delay is higher than BEB's (e.g. 0.94 vs 0.77 ms at 20 Mb/s offered): the load factor shrinks the window, but not all the way to CWmin.
- Each station must keep a log of overheard addresses and run a small optimisation table — cheap, but more state than BEB's single counter.
- The optimal-τ table is computed from the analytical model and assumes known Ts and Tc; with multi-rate stations, frames have different durations and the target is only approximate.

- Legacy BEB stations take a larger share of the channel than CEAB stations (about 2.6 : 1).

Conditions where the proposed scheme fails or gives no benefit

- **RTS/CTS with few stations:** collisions are cheap and BEB is already near the optimum; CEAB is equal or marginally lower (−0.3 % at N = 5).
- **High frame-error rates (FER ≥ 0.2):** losses dominate and every scheme converges; EIED is slightly better at FER = 0.3.
- **Hidden terminals:** the estimator assumes that every station hears every frame. Stations outside each other's range would count different senders, and collisions at the AP would not be reflected in the estimate (not simulated; expected to under-estimate contention).
- **Very short-lived bursts:** a station that has just joined has heard no frames yet and starts at CWmin, so for the first tens of milliseconds it behaves like BEB.
- **Light load:** with only a few active stations BEB is already close to optimal (S1 gain only 1.9 %); the benefit of CEAB is a dense-network benefit.

6. CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

A validated event-driven simulator of the IEEE 802.11g DCF was built and used to compare nine CSMA/CA waiting strategies over 2170 independent runs. The proposed CEAB scheme estimates the number of contending stations from overheard transmitter addresses and holds the contention window at the analytically optimal value instead of resetting it after each success. Key quantified findings:

- Throughput: +1.9 % (N = 5), +15.8 % (N = 20), +33.5 % (N = 50) and +56.7 % (N = 100) over BEB; within 2.5 % of the theoretical limit.
- Collision probability at N = 50: 0.595 → 0.204; drop rate 2.0 % → 0 %.
- 99th-percentile access delay at N = 50: 304 ms → 65 ms (79 % lower).
- Short-term Jain fairness at N = 50: 0.397 → 0.910.
- Maximum carried load with Poisson traffic: 24.4 → 29.9 Mb/s; light-load delay unchanged.
- The estimator tracks membership changes within about 150 ms.
- Among the history-based rules, only those that keep contention information across frames (EIED, MILD) scale; linear, Fibonacci and constant windows are worse than BEB at high N.

6.2 Limitations

- Simulation only, in a single collision domain with one PHY rate; no hidden terminals, capture effect or multi-rate stations.
- Traffic is uplink only; no TCP dynamics, EDCA priorities or frame aggregation (A-MPDU).
- The optimal window is derived from an analytical model that assumes saturated, homogeneous stations.
- Idle Sense was re-implemented from its published description in simplified form; the original may perform slightly differently.

6.3 Future enhancements

- Extend the simulator to multiple overlapping cells and hidden terminals, and estimate contention from both decoded addresses and unexplained busy periods.

- Make CEAB airtime-aware for multi-rate networks (count airtime per sender rather than frames).
- A coexistence mode that detects legacy stations (from their short post-success gaps) and lowers the window floor to recover a fair share.
- Port the algorithm to ns-3 and to open firmware (e.g. OpenWiFi or ath9k) for a testbed experiment.
- Apply the estimator to 802.11ah / IoT networks with hundreds of stations, where the gains should be larger still.

7. REFERENCES

[1] IEEE Standard for Information Technology—Telecommunications and Information Exchange between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications, IEEE Std 802.11-2020, Feb. 2021.

[2] G. Bianchi, “Performance analysis of the IEEE 802.11 distributed coordination function,” IEEE J. Sel. Areas Commun., vol. 18, no. 3, pp. 535–547, Mar. 2000.

[3] F. Calì, M. Conti, and E. Gregori, “Dynamic tuning of the IEEE 802.11 protocol to achieve a theoretical throughput limit,” IEEE/ACM Trans. Netw., vol. 8, no. 6, pp. 785–799, Dec. 2000.

[4] V. Bharghavan, A. Demers, S. Shenker, and L. Zhang, “MACAW: A media access protocol for wireless LANs,” in Proc. ACM SIGCOMM, London, U.K., 1994, pp. 212–225.

[5] N.-O. Song, B.-J. Kwak, J. Song, and L. E. Miller, “Enhancement of IEEE 802.11 distributed coordination function with exponential increase exponential decrease backoff algorithm,” in Proc. IEEE Veh. Technol. Conf. (VTC-Spring), 2003, pp. 2775–2778.

[6] M. Heusse, F. Rousseau, R. Guillier, and A. Duda, “Idle Sense: An optimal access method for high throughput and fairness in rate diverse wireless LANs,” in Proc. ACM SIGCOMM, Philadelphia, PA, USA, 2005, pp. 121–132.

[7] G. Bianchi and I. Tinnirello, “Kalman filter estimation of the number of competing terminals in an IEEE 802.11 network,” in Proc. IEEE INFOCOM, San Francisco, CA, USA, 2003, pp. 844–852.

[8] B.-J. Kwak, N.-O. Song, and L. E. Miller, “Performance analysis of exponential backoff,” IEEE/ACM Trans. Netw., vol. 13, no. 2, pp. 343–355, Apr. 2005.

[9] R. Jain, D.-M. Chiu, and W. R. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” Digital Equipment Corp., Tech. Rep. DEC-TR-301, 1984.

[10] R. M. Metcalfe and D. R. Boggs, “Ethernet: Distributed packet switching for local computer networks,” Commun. ACM, vol. 19, no. 7, pp. 395–404, Jul. 1976.

[11] L. Kleinrock and F. A. Tobagi, “Packet switching in radio channels: Part I—Carrier sense multiple-access modes and their throughput-delay characteristics,” IEEE Trans. Commun., vol. COM-23, no. 12, pp. 1400–1416, Dec. 1975.

[12] Q. Ni, I. Aad, C. Barakat, and T. Turletti, “Modeling and analysis of slow CW decrease for IEEE 802.11 WLAN,” in Proc. IEEE Int. Symp. Personal, Indoor and Mobile Radio Commun. (PIMRC), 2003, pp. 1717–1721.

[13] H. Wu, Y. Peng, K. Long, S. Cheng, and J. Ma, “Performance of reliable transport protocol over IEEE 802.11 wireless LAN: Analysis and enhancement,” in Proc. IEEE INFOCOM, 2002, pp. 599–607.

[14] S. Manaseer, M. Ould-Khaoua, and L. M. Mackenzie, “Fibonacci backoff algorithm for mobile ad hoc networks,” in Proc. 7th Annu. PostGraduate Symp. Convergence of Telecommun., Netw. and Broadcasting (PGNet), Liverpool, U.K., 2006.

[15] J. Håstad, T. Leighton, and B. Rogoff, “Analysis of backoff protocols for multiple access channels,” SIAM J. Comput., vol. 25, no. 4, pp. 740–774, 1996.

[16] E. H. Simpson, “Measurement of diversity,” Nature, vol. 163, p. 688, Apr. 1949.

[17] M. O. Hill, “Diversity and evenness: A unifying notation and its consequences,” Ecology, vol. 54, no. 2, pp. 427–432, Mar. 1973.

APPENDIX A: IMPORTANT SOURCE CODE LISTINGS

A.1 CEAB backoff strategy (csmasim/strategies.py)

```
class CEAB(Strategy):
    """Contender-Estimation Adaptive Backoff (proposed).


    1. Contender estimation from overheard frames. Every station decodes the transmitter
       address (TA) of each successful frame on the channel and keeps the last M of them.
       With c_j = frames from station j in that window, the effective number of contenders is
       the inverse Simpson index (unbiased form)
               n_eff = M (M-1) / sum_j c_j (c_j - 1),
       which equals n when n stations share the channel equally and discounts lightly-loaded
       stations. It uses no model of the other stations' backoff rule.
    2. Load factor. Each station keeps an EWMA of the idle gap between busy periods, I_obs (us),
       and compares it with I*, the mean idle gap at the analytical throughput optimum:
               rho = min(1, I* / I_obs),   n_c = max(1, rho * n_eff).
       Under saturation I_obs ~ I* and rho ~ 1; with unsaturated traffic the channel is idle
       far longer than at the optimum, rho << 1, and the window shrinks towards CWmin. The loop
       is self-correcting: a window that is too large creates extra idle time, which lowers rho.
    3. Target window W* = 2/tau*(n_c) - 1, tau*(n) = optimal attempt probability for the PHY
       timing (Ts, Tc, slot); the base window W_base moves a fraction gamma towards W*.
    4. Optional bounded escalation: the k-th retry of a frame uses W_base * 2^min(k, K) and a
       success returns to W_base, never to CWmin. K=0 (default) means collisions do not change
       the window at all -- the estimate alone sets it (see the ablation study).
    """
    name, label = "ceab", "CEAB (proposed)"


    def __init__(self, n, phy, M=200, beta_u=0.05, gamma=0.5, K=0, w_floor=None, **kw):
        super().__init__(n, phy)
        self.M, self.beta_u, self.gamma, self.K = int(M), beta_u, gamma, K
        self.w_floor = float(w_floor if w_floor is not None else phy.cw_min)
        self.W_base = np.full(n, self.wmin)
        self.log = []                               # TA of every successful frame heard
        self.join = np.zeros(n, dtype=np.int64)     # log position when the station (re)joined
        self.n_total = kw.get("n_total", n)
        self.idle = np.full(n, (self.wmin - 1) / 2 * phy.slot)   # EWMA idle gap (us)
        self.idle_star = optimal_idle_slots(phy) * phy.slot       # idle gap at the optimum (us)


    def n_eff(self, i):
        lo = max(int(self.join[i]), len(self.log) - self.M)
        win = self.log[lo:]
        m = len(win)
        if m < 2:
            return 1.0
        c = np.bincount(win)
        d = float((c * (c - 1)).sum())
        return m * (m - 1) / d if d > 0 else float(m)


    def load_factor(self, idx=slice(None)):
        return np.minimum(1.0, self.idle_star / np.maximum(self.idle[idx], 1e-9))


    def n_hat(self, idx=None):
        idx = np.arange(self.n) if idx is None else np.atleast_1d(idx)
        ne = np.array([self.n_eff(i) for i in idx])
        return np.maximum(1.0, self.load_factor(idx) * ne)


    def observe(self, idle, tx_mask, contending_mask, success, src=-1, busy_us=0.0, idle_us=0.0):
        if success and src >= 0:
            self.log.append(int(src))
            if len(self.log) > 4 * self.M:      # trim history nobody needs any more
                cut = len(self.log) - self.M
                self.log = self.log[cut:]
                self.join = np.maximum(self.join - cut, 0)
        b = self.beta_u
        self.idle = (1 - b) * self.idle + b * idle_us


    def _update_base(self, idx):
        target = w_opt(self.n_hat(idx), self.phy)
        wb = self.W_base[idx]
        self.W_base[idx] = np.clip(wb + self.gamma * (target - wb), self.w_floor, self.wmax)


    def on_success(self, idx):
```

```
        self._update_base(idx)
        self.W[idx] = self.W_base[idx]


    def on_failure(self, idx, retry):
        self._update_base(idx)
        k = np.minimum(retry, self.K)
        self.W[idx] = np.minimum(self.W_base[idx] * 2.0 ** k, self.wmax)


    def on_drop(self, idx):
        self.W[idx] = self.W_base[idx]


    def reset(self, idx):
        self.W[idx] = self.wmin
        self.W_base[idx] = self.wmin
        self.join[idx] = len(self.log)
        self.idle[idx] = (self.wmin - 1) / 2 * self.phy.slot
```

A.2 Baseline BEB and the strategy interface (csmasim/strategies.py)

```
class Strategy:
    name = "base"
    label = "Base"


    def __init__(self, n: int, phy: PhyParams, **kw):
        self.n = n
        self.phy = phy
        self.wmin = float(phy.cw_min)
        self.wmax = float(phy.cw_max)
        self.W = np.full(n, self.wmin)


    def window(self, idx):
        return np.clip(np.rint(self.W[idx]), 1, self.wmax).astype(np.int64)


    def on_success(self, idx):
        self.W[idx] = self.wmin


    def on_failure(self, idx, retry):
        pass


    def on_drop(self, idx):
        self.W[idx] = self.wmin


    def observe(self, idle, tx_mask, contending_mask, success, src=-1, busy_us=0.0, idle_us=0.0):
        pass


    def reset(self, idx):
        self.W[idx] = self.wmin



class BEB(Strategy):
    """IEEE 802.11 DCF binary exponential backoff (baseline)."""
    name, label = "beb", "BEB (802.11)"


    def on_failure(self, idx, retry):
        self.W[idx] = np.minimum(self.W[idx] * 2, self.wmax)
```

A.3 Core of the channel engine: one transmission event (csmasim/simulator.py)

```
            # ---- a transmission starts at t_tx
            self.counter[cont] -= m
            idle_acc += m
            now = t_tx
            tx = cont[self.counter[cont] == 0]
            measuring = now >= T0
            success = tx.size == 1 and (cfg.fer == 0 or rng.random() >= cfg.fer)
            busy = phy.ts if success else phy.tc
            if measuring:
                attempts += tx.size
                busy_events += 1
```

```
                if tx.size > 1:
                    busy_coll_events += 1
                    failures_coll += tx.size
                elif not success:
                    failures_err += 1


            if len(trace) < cfg.trace_limit:
                trace.append({"t_us": now, "idle_slots": idle_acc, "tx": [int(i) for i in tx],
                              "W": [int(self.groups[self.group_of[i]][0].window(self.local_of[i]))
                                    for i in tx],
                              "retry": [int(retry[i]) for i in tx],
                              "outcome": "SUCCESS" if success else
                              ("COLLISION" if tx.size > 1 else "FRAME-ERROR"),
                              "busy_us": busy})
            if self.observers:
                tx_mask = np.zeros(n, dtype=bool)
                tx_mask[tx] = True
                cont_mask = self.counter >= 0
                for g in self.observers:
                    strat, idx = self.groups[g]
                    strat.observe(idle_acc, tx_mask[idx], cont_mask[idx], success,
                                  int(tx[0]) if success else -1, busy,
                                  now - last_busy_end)
            idle_acc = 0
            now += busy
            last_busy_end = now


            for i in tx:
                if success:
                    if measuring:
                        succ_t.append(now)
                        succ_node.append(i)
                        acc_delay.append(now - hol[i])
                        if not saturated:
                            e2e_delay.append(now - queues[i][0])
                    self._call(i, "on_success")
                    retry[i] = 0
                    if saturated:
                        hol[i] = now
                        self._draw(i)
                    else:
                        queues[i].popleft()
                        if queues[i]:
                            hol[i] = now
                            self._draw(i)
                        else:
                            self.counter[i] = -1
                else:
                    retry[i] += 1
                    if phy.retry_limit is not None and retry[i] > phy.retry_limit:
                        if measuring:
                            drops += 1
                        self._call(i, "on_drop")
                        retry[i] = 0
                        if not saturated:
                            queues[i].popleft()
                            if not queues[i]:
                                self.counter[i] = -1
                                continue
                        hol[i] = now
                        self._draw(i)
                    else:
                        self._call(i, "on_failure", retry[i])
                        self._draw(i)
```

A.4 Optimal attempt probability table (csmasim/analytical.py)

```
def throughput_from_tau(tau: float, n: float, phy: PhyParams) -> float:
    """Saturation throughput (Mb/s) of n stations each attempting with probability tau per slot."""
    p_idle = (1 - tau) ** n
    p_tr = 1 - p_idle
    if p_tr <= 0:
        return 0.0
    p_s = n * tau * (1 - tau) ** (n - 1) / p_tr
    num = p_s * p_tr * phy.payload * 8
    den = p_idle * phy.slot + p_tr * p_s * phy.ts + p_tr * (1 - p_s) * phy.tc
    return num / den  # bits per microsecond == Mb/s
```

```
def w_opt(n_hat, phy: PhyParams):
    """Window size W (backoff uniform on [0, W-1]) whose mean attempt rate equals tau*: tau =
2/(W+1)."""
    return 2.0 / tau_opt(n_hat, phy) - 1.0
```

APPENDIX B: SCREENSHOTS, EVENT TRACES AND RAW RESULT TABLES

B.1 Interactive demo (python demo.py gui). Sliders change N, offered load, frame error rate, payload and duration; check-boxes choose strategies; the radio button selects basic or RTS/CTS access.



Figure B.1: GUI of the live demo comparing BEB, Idle Sense and CEAB with 50 stations.

B.2 Command-line comparison (python demo.py compare -n 50 --runs 3).

```
N=50  traffic=saturated  FER=0.0  access=basic  payload=1500 B  duration=5.0s x 3 run(s)

Metric                                BEB (802.11)       CEAB (proposed)       Δ last vs first
-------------------------------------------------------------------------------------------------
Throughput (Mb/s)                      22.35 ± 0.02          29.91 ± 0.08               +33.9%
Collision probability                 0.595 ± 0.001         0.198 ± 0.004               +66.6%
Mean access delay (ms)                 20.27 ± 0.47          20.06 ± 0.04                +1.1%
p99 access delay (ms)                 307.18 ± 3.76          63.90 ± 1.16               +79.2%
Mean end-to-end delay (ms)                     n/a                   n/a
Short-term Jain fairness              0.396 ± 0.012         0.911 ± 0.001              +130.2%
Drop rate                          0.0195 ± 0.0017       0.0000 ± 0.0000              +100.0%
(Δ is the improvement of the last strategy over the first; positive = better)
```

B.3 Channel event trace (python demo.py trace -n 5 --events 15). The simulator equivalent of a packet capture: each line is one busy period with the idle slots before it, the outcome, the transmitting stations, the window each one drew its backoff from and its retry count. Under BEB a station returns to W = 16 after every success; under CEAB the windows grow to the estimated optimum (≈ 40 for 5 stations) and stay there.

```
BEB (802.11): first 15 busy periods, N=5
  #  time (us)  idle  outcome     stations       W used         retry      busy (us)
  1          0     0  SUCCESS     S5             16             0                314
  2        332     2  SUCCESS     S5             16             0                314
  3        691     5  SUCCESS     S1             16             0                314
  4       1014     1  SUCCESS     S2             16             0                314
  5       1355     3  SUCCESS     S2             16             0                314
  6       1678     1  SUCCESS     S3             16             0                314
  7       2019     3  COLLISION   S2,S4,S5       16,16,16       0,0,0            323
  8       2405     7  SUCCESS     S1             16             0                314
  9       2728     1  SUCCESS     S4             32             1                314
 10       3060     2  SUCCESS     S3             16             0                314
 11       3383     1  SUCCESS     S1             16             0                314
 12       3715     2  SUCCESS     S2             32             1                314
 13       4038     1  COLLISION   S2,S4          16,16          0,0              323
 14       4361     0  SUCCESS     S2             32             1                314
 15       4720     5  SUCCESS     S1             16             0                314

CEAB (proposed): first 15 busy periods, N=5
  #  time (us)  idle  outcome     stations       W used         retry      busy (us)
  1          0     0  SUCCESS     S5             16             0                314
  2        332     2  SUCCESS     S5             16             0                314
  3        691     5  SUCCESS     S1             16             0                314
  4       1014     1  SUCCESS     S2             16             0                314
  5       1364     4  SUCCESS     S3             16             0                314
  6       1687     1  SUCCESS     S2             23             0                314
  7       2019     2  COLLISION   S4,S5          16,16          0,0              323
  8       2405     7  COLLISION   S1,S3,S5       16,35,29       0,0,1            323
  9       2773     5  SUCCESS     S4             29             1                314
 10       3123     4  SUCCESS     S3             38             1                314
 11       3482     5  SUCCESS     S5             35             2                314
 12       3823     3  SUCCESS     S5             38             0                314
 13       4137     0  SUCCESS     S5             35             0                314
 14       4460     1  SUCCESS     S2             32             0                314
 15       4828     6  SUCCESS     S1             29             1                314
```

B.4 Raw per-run results, scenario S3 (N = 50, saturated). The complete per-run data (results/raw_runs.csv), the summary (results/summary.csv) and all figures are produced by run_experiments.py and are submitted with the source code.

Table B.1: Individual runs behind the S3 row of Table 5.3.

Seed	BEB thr (Mb/s)	CEAB thr (Mb/s)	BEB coll. prob.	CEAB coll. prob.	BEB p99 (ms)	CEAB p99 (ms)
1	22.307	29.847	0.5953	0.2023	307.2	65.2
2	22.347	29.797	0.5954	0.2050	303.0	64.7
3	22.361	29.885	0.5937	0.2010	308.9	63.9
4	22.304	29.883	0.5964	0.2006	297.9	64.4
5	22.345	29.709	0.5944	0.2088	305.0	65.9
6	22.351	29.829	0.5937	0.2036	302.0	65.4
7	22.360	29.768	0.5937	0.2052	305.1	64.4
8	22.203	29.825	0.5981	0.2018	308.2	63.6
9	22.327	29.685	0.5951	0.2103	292.5	66.2
10	22.420	29.892	0.5915	0.1997	310.7	63.9

B.5 Validation data: simulated BEB vs. Bianchi model (unlimited retries).

Table B.2: Simulator validation.

N	Model thr (Mb/s)	Sim thr (Mb/s, mean ± SD)	Error (%)	Model p	Sim p
2	32.31	31.82 ± 0.07	-1.53	0.105	0.110
5	30.48	30.05 ± 0.07	-1.42	0.272	0.265
10	28.31	28.17 ± 0.06	-0.49	0.384	0.366
20	26.03	26.05 ± 0.07	+0.09	0.481	0.459
35	24.09	24.23 ± 0.10	+0.55	0.552	0.529
50	22.79	23.02 ± 0.06	+1.00	0.595	0.572

N	Model thr (Mb/s)	Sim thr (Mb/s, mean ± SD)	Error (%)	Model p	Sim p
75	21.21	21.53 ± 0.07	+1.52	0.644	0.621
100	20.02	20.37 ± 0.05	+1.75	0.678	0.656